

ServoraBd

Local Service Marketplace

ENGINEERING & INTERVIEW GUIDE

A complete walkthrough of the system — how to run it, how every layer works, why each decision was made, and how to defend all of it in a technical interview.

Version	1.0
Status	Complete through M10
Date	29 September 2026
Author	Jahid H. R.
Stack	Django · DRF · React (JS) · PostgreSQL

Contents

1	How to read this guide
2	The domain in plain language
3	Setting up and running the project
4	Architecture and code structure
5	Accounts: identity, OTP and JWT
6	Catalogue: services and the location tree
7	Providers: offerings, coverage, hours and verification
8	The trust engine
9	Bookings: the request and job lifecycle
10	Reviews and payments
11	Operations, scheduled work and security
12	The frontend
13	Deployment and CI
14	Interview preparation: how to present this
15	Question bank with model answers
16	Quick reference

1. How to read this guide

This document explains the ServoraBd codebase from the outside in. It is written for one reader: the person who built it and now has to explain it under questioning. Everything here is traceable to a real file in the repository, and file paths are given so you can open the code alongside the prose.

1.1 The three questions this guide answers

Question	Where it is answered
How do I get it running on a fresh machine?	Section 3. Setup, database, seeding, both servers.
How does the code actually work, end to end?	Sections 4–12. Architecture, each backend app, the trust engine, the frontend, security, operations.
How do I defend it in an interview?	Sections 13–16. Question bank with model answers, the weaknesses an interviewer will find, and a 90-second pitch.

1.2 What the project is, in one paragraph

ServoraBd is a two-sided marketplace for home services — electricians, plumbers, AC technicians — in Dhaka. A customer posts a request; providers in that area who offer that service see it and can accept; accepting creates a booking that moves through a state machine to completion, payment and a double-blind review. The distinguishing feature is not the booking flow, which is a solved problem. It is the **trust score**: a six-factor, statistically smoothed, time-decayed measure of a provider's reliability, computed from recorded platform events rather than self-report, and shown to customers broken down factor by factor.

The one thing to lead with

Star ratings are broken. A provider with one 5-star review outranks one with two hundred jobs at 4.8, and ratings only measure jobs that were *completed* — a provider who accepts ten bookings and abandons eight can hold a perfect score. ServoraBd measures what star ratings cannot: whether the person finishes what they start.

1.3 The headline numbers

Measure	Value
Backend tests	569, passing
Frontend tests	47 unit and component, plus Playwright E2E
Django apps	9 (common, operations, accounts, catalogue, providers, trust, bookings, reviews, payments)
API endpoints	60+ under /api/v1/
Provider search p95, 10,018 providers	100 ms (target 400 ms)
Nightly trust recompute, 10,018 providers	175 s (target 15 min)
Landing page LCP, Slow 4G, 4× CPU throttle	1.58 s (target 2.5 s)
Accessibility	axe-core: zero WCAG 2.1 AA violations
Known vulnerabilities	0 (pip-audit, npm audit, in CI)

2. The domain in plain language

Before any code, understand the nouns. Almost every interview question resolves to one of these eight concepts, and being fluent in them makes every later answer shorter.

2.1 The actors

Actor	What they do	How the code models them
Customer	Posts service requests, confirms completion, pays cash, writes reviews.	<code>CustomerProfile</code> , one-to-one with <code>User</code> .
Provider	Lists services and prices, picks service areas and hours, accepts requests, does the job, gets a trust score.	<code>ProviderProfile</code> , one-to-one with <code>User</code> .
Admin	Reviews verification documents, resolves flagged payments, hides abusive reviews, watches scheduled jobs.	<code>User.is_staff</code> ; Django admin plus admin API routes.
System	Expires unanswered requests, auto-confirms stale jobs, reveals reviews, recomputes trust nightly.	Management commands run by cron; actor recorded as <code>system</code> in the event log.

2.2 The core objects

- **Service** — a thing that can be bought, e.g. 'Ceiling fan installation'. Belongs to a **ServiceCategory**. Carries a suggested price band so the UI can flag a quote as low, normal or high.
- **Location** — a three-level self-referential tree: city → thana → area. A provider's coverage is stored at thana or area level; a search on a thana matches providers registered in any of its areas, and vice versa.
- **ServiceRequest** — a customer asking for work. Either *broadcast* (every eligible provider in the area sees it) or *direct* (aimed at one provider). Expires unanswered after 24 hours.
- **Booking** — created the moment a provider accepts a request. Holds the agreed price and moves through a seven-state machine. Every transition is written to an append-only **BookingEvent** log.
- **Payment** and **LedgerEntry** — cash settlement. Both sides record the amount; if they disagree the payment is flagged for an admin instead of being silently trusted.
- **Review** — double-blind. Neither side sees the other's words until both have submitted, or 14 days pass.
- **TrustSnapshot** — an append-only record of a provider's score every time it was recomputed, including the full factor breakdown and which event triggered it. This is the audit trail.

2.3 The happy path, end to end

- 1 A customer registers with a Bangladeshi phone number and receives a six-digit OTP.
- 2 They browse services, search providers, and compare up to three side by side on trust rather than price.
- 3 They post a request through a six-step wizard: service, where, problem, when, who, confirm.
- 4 Every provider who offers that service, covers that area, is accepting work and is not under review has `requests_received` incremented — the denominator of the responsiveness factor.
- 5 A provider accepts. The request closes, a booking is created at the agreed price, and the provider's `jobs_accepted` goes up.
- 6 The provider starts the job, then completes it with a final price.
- 7 The customer confirms. That single call increments `jobs_completed`, records the payment, writes three ledger entries, and queues a trust recompute.

- 8 Both sides review each other. When the second one lands, both are published at once and trust is recomputed again.

Why the counters matter

`requests_received`, `jobs_accepted`, `jobs_completed` and `jobs_cancelled` on `ProviderProfile` are denormalised counters maintained with atomic `F()` expressions. They are the raw inputs to the trust engine, which is why trust can be recomputed for ten thousand providers in under three minutes rather than by walking every booking.

3. Setting up and running the project

3.1 What you need installed

Tool	Version	Why
Python	3.12	Django 5.2 LTS targets it; CI pins it.
PostgreSQL	16 or newer	No extensions needed. CI runs 18. Check constraints and partial unique indexes are used, so SQLite is not a substitute.
Node	20 or newer	Vite 6 and the React toolchain. CI runs 24.
Git	any	Cloning; the repo uses <code>.gitattributes</code> for line endings.

3.2 Database first

Run these once as the postgres superuser. The third line is the one people miss: since PostgreSQL 15 a non-owner role cannot create tables in the public schema, so `migrate` fails with a permission error without it.

```
psql -U postgres -c "CREATE DATABASE servorabd;"
psql -U postgres -c "CREATE USER sheba WITH PASSWORD 'your-password' CREATEDB;"
psql -U postgres -d servorabd -c "ALTER SCHEMA public OWNER TO sheba;"
```

The `CREATEDB` privilege is not optional. `pytest` creates and drops a `test_servorabd` database on every run; without it the whole suite refuses to start.

3.3 Backend

```
cd backend
python -m venv venv
venv\Scripts\activate          # macOS/Linux: source venv/bin/activate
pip install -r requirements.txt
copy .env.example .env         # then edit it
python manage.py migrate
python manage.py seed_catalogue # 8 categories, 45 services, 38 locations
python manage.py seed_demo      # optional: 18 providers with real history
python manage.py createsuperuser
python manage.py runserver
```

The API is then at `http://127.0.0.1:8000/api/v1/` and Django admin at `/admin/`.

What goes in `.env`

Variable	Meaning
<code>SECRET_KEY</code>	Required, no default. Also salts the OTP hash, so changing it invalidates every outstanding code.
<code>DATABASE_URL</code>	e.g. <code>postgres://sheba:pw@localhost:5432/servorabd</code>
<code>DEBUG</code>	Defaults to False. The dev settings module forces it True.
<code>OTP_TTL_SECONDS</code>	Default 300 (five minutes).
<code>OTP_MAX_SENDS_PER_HOUR</code>	Default 3 per phone number.
<code>OTP_MAX_VERIFY_ATTEMPTS</code>	Default 5 per code.

Variable	Meaning
JWT_ACCESS_MINUTES	Default 15.
JWT_REFRESH_DAYS	Default 14.
PLATFORM_COMMISSION_PERCENT	Default 12.
TRUSTED_PROXY_COUNT	Default 0. Critical security setting — see section 11.2.
USE_OBJECT_STORAGE	Production only. Switches file storage to S3-compatible object storage.

3.4 Frontend

In a second terminal, with the backend already running:

```
cd frontend
npm install
npm run dev # http://127.0.0.1:5173
```

Vite proxies /api and /media to `http://127.0.0.1:8000`, so the browser sees a single origin and there is no CORS configuration to get wrong in development. Point it elsewhere with `BACKEND_URL=http://127.0.0.1:8100 npm run dev`.

3.5 Logging in during development

Where is the OTP?

There is no SMS provider wired up. In development `OTP_ECHO_TO_LOG` is `True`, so the code is printed to the runserver console: look for `OTP for +8801... is 123456`. In production that flag is `False` and the log records only that a code was issued — never the code itself.

3.6 Every command you might need

Command	What it does
<code>python -m pytest</code>	All 569 backend tests, about two minutes.
<code>python -m pytest -k otp</code>	One area by keyword.
<code>python manage.py seed_catalogue</code>	Idempotent: categories, services, the Dhaka location tree.
<code>python manage.py seed_demo</code>	18 providers across archetypes, with bookings, reviews and cancellations, so the trust engine has real data to chew on.
<code>python manage.py benchmark</code>	Times search, provider detail, trust breakdown and the nightly recompute against the PRD targets.
<code>python manage.py run_scheduled_jobs</code>	Runs every due scheduled job once — the local stand-in for cron.
<code>python manage.py crontab</code>	Prints the crontab lines to install on a server.
<code>python manage.py check_proxy_count <url></code>	Probes a live deployment for the forged-header rate-limit bypass (section 11.2).
<code>npm run lint</code>	ESLint, zero warnings allowed.
<code>npm test</code>	Vitest unit and component tests.

Command	What it does
<code>npm run e2e</code>	Playwright: both golden paths and the accessibility scan, Chromium at 360 px.
<code>npm run perf:lcp</code>	Production build, then LCP under Slow 4G and Fast 4G throttling.

Why the test settings module exists

`config/settings/test.py` swaps in a fast password hasher. Production-strength PBKDF2 costs about a second per hash, and with a user or two created per test that alone took the suite from two minutes to fourteen. This is a standard Django technique and a good thing to mention unprompted — it shows you profiled your own test suite.

4. Architecture and code structure

4.1 The shape of the repository

```

Local-Service/
|-- backend/           Django project
|   |-- config/
|   |   |-- settings/   base.py, dev.py, prod.py, test.py
|   |   |-- urls.py     mounts every app under /api/v1/
|   |   +-- wsgi.py, asgi.py
|   |-- apps/
|   |   |-- common/     base models, exceptions, pagination, throttling, storage
|   |   |-- operations/ scheduled-job registry and run log
|   |   |-- accounts/   User, profiles, OTP, JWT, roles
|   |   |-- catalogue/  categories, services, location tree
|   |   |-- providers/  offerings, areas, availability, verification, search
|   |   |-- trust/      the six-factor engine and snapshots
|   |   |-- bookings/   requests, responses, bookings, state machine
|   |   |-- reviews/   double-blind reviews and replies
|   |   +-- payments/   cash settlement, commission, ledger
|   +-- manage.py, requirements.txt, confest.py
|-- frontend/          React 18 + Vite client
|-- docs/              PRD (PDF + generator), trust math reference
|-- deploy/            Nginx, gunicorn, systemd, backup scripts, guides
|-- render.yaml        Render blueprint
+-- .github/workflows/ CI

```

4.2 The layered pattern every app follows

This is the single most important structural fact about the backend, and the thing that will most impress an interviewer. Every app is split the same way, and each layer has exactly one job.

File	Responsibility	Rule
models.py	Database shape, constraints, indexes, and small derived properties.	No business logic that spans more than one object.
selectors.py	Read queries. Everything the API needs to <i>show</i> .	Never writes. Owns the <code>select_related</code> / <code>prefetch_related</code> that kills N+1.
services.py	Write operations and business rules. The only place state changes.	Three hard rules — see below.
serializers.py	Input validation and output shaping.	Validation of <i>format</i> ; services own validation of <i>meaning</i> .
views.py	HTTP: parse the request, check permission, call one service, return a response.	Thin. A view that contains an <code>if</code> about business rules is a bug.
permissions.py	Who may call this at all.	Role checks, not object checks.

The three service-layer rules

These come from PRD §8.5 and are load-bearing. They exist so that moving any service call onto a task queue later is a one-line change at the call site rather than a rewrite.

- Services take IDs, not model instances.** A queue message has to be JSON-serialisable; a Django model instance is not. Every service signature in the codebase is `def thing(*, booking_id, provider_id, ...)`.

- 2 **Services never touch** request. A background worker does not have one. The view extracts `request.user.id` and passes it down; the service never reaches up.
- 3 **Services are idempotent.** A retried job or a double-fired cron must be harmless. `record_completion_payment` returns the existing payment if one is already there rather than creating a second.

Interview gold

Being able to say 'every service takes IDs rather than instances so that adopting Celery later is one line per call site, and here is the PRD section that defines the load thresholds that would justify it' demonstrates architectural thinking, not just Django familiarity. It also pre-empts the obvious question, 'why no Celery?'

4.3 Keyword-only arguments everywhere

Every service function uses the bare `*` marker, forcing callers to name every argument. The reason is concrete: `cancel_booking(booking_id, customer_id, provider_id)` has three integer parameters, and getting them in the wrong order would silently cancel the wrong person's job. Naming them makes that class of bug impossible.

4.4 The two abstract base models

In `apps/common/models.py`:

- **TimeStampedModel** — gives every table `created_at` (indexed) and `updated_at`.
- **AppendOnlyModel** — overrides `save()` to raise if `self.pk` is already set, and `delete()` to always raise. Used by `BookingEvent`, `TrustSnapshot`, `LedgerEntry` and `ReviewEdit`.

The append-only base is the audit story in four lines of code. When an interviewer asks how you would prove a provider's score was not quietly edited, the answer is that the model physically refuses the write — it is not a convention anyone has to remember.

4.5 The error envelope

Every error the API returns, from a validation failure to an unhandled exception, comes back in one shape. This is enforced by a custom DRF exception handler in `apps/common/exceptions.py`, registered once in settings.

```
{
  "error": {
    "code": "invalid_state_transition",
    "message": "A booking in state 'completed' cannot move to 'in_progress'.",
    "details": {"from_state": "completed", "to_state": "in_progress"}
  }
}
```

The domain exceptions form a small hierarchy, each carrying its own HTTP status, so a service can raise a meaning and the HTTP layer translates it without a single `try/except` in any view:

Exception	Status	Used for
<code>DomainError</code>	400	Base class. A business rule said no.
<code>InvalidStateTransition</code>	409	The state machine refused. 409 Conflict, not 400, because the request was well-formed — the resource was simply in the wrong state.
<code>RateLimited</code>	429	OTP send or verify budget exhausted.

Exception	Status	Used for
VerificationError	400	Wrong, expired or missing OTP.
NotFound	404	No such object.

The handler also catches the case DRF returns None for — a genuinely unhandled exception — logs it with a stack trace, and returns a generic 500 envelope. The client never sees an internal error message, and the frontend never has to parse two different error formats.

5. Accounts: identity, OTP and JWT

backend/apps/accounts/. This app owns the custom user model, phone verification, login, and the dual-role system. It is the app most likely to be probed in an interview because authentication is where security bugs live.

5.1 The custom User model

User extends AbstractBaseUser and PermissionsMixin. There is no username; USERNAME_FIELD is phone. Both phone and email are nullable and unique, with a database-level check constraint guaranteeing at least one is present:

```
models.CheckConstraint(
    condition=models.Q(phone__isnull=False)
    | models.Q(email__isnull=False),
    name="user_has_phone_or_email",
)
```

That constraint is the answer to ‘what stops a user row with neither identifier?’ — the database does, not the application. Serializer validation can be bypassed by a management command or a shell session; a check constraint cannot.

5.2 Phone normalisation

Bangladeshi numbers get typed six different ways. normalise_bd_phone() in validators.py funnels them all into one canonical form before anything touches the database.

User types	Stored as
01712345678	+8801712345678
8801712345678	+8801712345678
+880 1712-345678	+8801712345678
1712345678	+8801712345678

The regex `^\+8801[3-9]\d{8}$` then validates the result. The `[3-9]` is not arbitrary: Bangladeshi mobile prefixes run 013 to 019, so 010 to 012 are rejected as impossible numbers rather than accepted and later found undeliverable.

Normalisation happens in three places, deliberately: in the serializer's PhoneField, in User.clean(), and in UserManager._create_user(). Belt and braces — a user created from a management command is normalised the same way as one created through the API, so the uniqueness constraint actually means something.

5.3 How OTP works

The PhoneOTP model never stores a code. It stores a SHA-256 hash of phone:code:SECRET_KEY.

```
@staticmethod
def hash_code(phone, code):
    payload = f'{phone}:{code}:{settings.SECRET_KEY}'.encode()
    return hashlib.sha256(payload).hexdigest()

def matches(self, code):
    return secrets.compare_digest(
        self.code_hash, self.hash_code(self.phone, code)
    )
```

Three details worth knowing by heart:

- 1 **The phone number is inside the hash.** Without it, a code issued for one number would verify against another — the hash would be identical.
- 2 **SECRET_KEY acts as a pepper.** An attacker with a dump of the OTP table cannot brute-force a million six-digit codes offline without also having the application secret.
- 3 `secrets.compare_digest`, **not** `==`. Constant-time comparison. A normal string compare short-circuits on the first differing byte, which leaks how much of the value was correct through response timing.

Codes are generated with `secrets.randbelow(1_000_000)` — the cryptographically secure generator, not random, whose Mersenne Twister state can be reconstructed from a handful of outputs.

The three rate limits on OTP

Limit	Default	Enforced where
Sends per phone per hour	3	<code>send_phone_otp</code> counts rows in the last hour — per <i>phone number</i> , so rotating IPs does not help.
Verify attempts per code	5	An attempts counter on the row, incremented inside <code>select_for_update</code> .
Requests per IP per hour	10 send / 30 verify	DRF <code>ScopedRateThrottle</code> at the view.

Two independent axes. The per-phone limit stops one attacker hammering one victim's number from many addresses; the per-IP limit stops one machine spraying many numbers. Either alone leaves a hole.

The locking subtlety in verify

`verify_phone_otp` uses two separate transactions on purpose. The first takes `select_for_update` on the OTP row, checks it, and on a mismatch increments attempts and *commits*. Only then is the exception raised, outside the transaction. If the increment and the raise shared one atomic block, the rollback would undo the increment — and the attempt counter would never move, giving an attacker unlimited guesses. This is a genuinely subtle bug and worth pointing to.

Sending a new code also consumes every outstanding unconsumed code for that phone and purpose, so a user cannot accumulate valid codes by pressing 'resend' repeatedly.

5.4 JWT authentication

SimpleJWT, configured in `config/settings/base.py`:

Setting	Value	Reason
<code>ACCESS_TOKEN_LIFETIME</code>	15 min	Short enough that a leaked token expires before it is much use.
<code>REFRESH_TOKEN_LIFETIME</code>	14 days	Long enough that a customer who books once a month is not logged out.
<code>ROTATE_REFRESH_TOKENS</code>	True	Every refresh issues a new refresh token.
<code>BLACKLIST_AFTER_ROTATION</code>	True	The old one is immediately invalid. Together with rotation this gives refresh-token reuse detection: a stolen token works once, and the moment either party uses the old one it fails.

The token payload is customised in `ServoraTokenObtainPairSerializer` to carry roles, `active_role` and `phone_verified`. This lets the frontend render the right navigation immediately without a second round trip. It is presentation only — every permission check on the server reads the database, never the claim.

A question you should expect

'You put roles in the JWT. What if someone edits them?' The token is signed with SECRET_KEY, so editing it breaks the signature. But the real answer is that it would not matter if they could: IsProvider checks `hasattr(user, 'provider_profile')` against the database on every single request. The claim is a UI hint, not an authorisation source.

5.5 The dual-role system

One human can be both a customer and a provider — an electrician still needs a plumber. Rather than duplicating accounts, one User can hold both profiles.

- `User.roles` is a computed property, derived from which profile objects exist plus `is_staff`. There is no `roles` column to drift out of sync.
- `active_role` is the UI mode the user is currently in. Switching it via `POST /me/switch-role/` is refused unless they actually hold that profile.
- Permission classes never consult `active_role`. `IsProvider` asks whether a `provider_profile` exists. A customer 'mode' does not remove a provider's access to provider endpoints — it only changes what the interface offers.

5.6 The custom authentication backend

`PhoneOrEmailBackend` lets a user log in with either identifier: if the string contains `@` it is looked up as an email, otherwise normalised as a phone number. One line in it matters disproportionately:

```
except User.DoesNotExist:
    User().set_password(password)
    return None
```

When the user does not exist, it hashes the password anyway and throws the result away. Without this, a login for a non-existent user returns in microseconds while one for a real user takes the full PBKDF2 cost — a timing oracle that lets an attacker enumerate which phone numbers are registered. This is the standard mitigation and Django's own backend does the same thing.

5.7 The login throttle

`LoginRateThrottle` in `apps/common/throttling.py` allows five attempts per 15 minutes per client. It exists as a custom class because DRF's rate string grammar only understands second, minute, hour and day — there is no way to express '5 per 15 minutes', so `parse_rate` is overridden to return the pair directly.

6. Catalogue: services and the location tree

backend/apps/catalogue/. Entirely public, entirely read-only through the API. Content is managed in Django admin and seeded by seed_catalogue.

6.1 Categories and services

ServiceCategory → Service, with slugs auto-generated on save. Slug uniqueness is scoped to the category, not global, so 'Installation' can exist under both Electrical and Plumbing.

Each service carries suggested_price_min and suggested_price_max, guarded by a check constraint that max is not below min. The price_flag() method turns a provider's quote into low, normal or high, which the UI renders as a badge. That is a small feature with a real purpose: it gives the customer price context without the platform setting prices.

6.2 The location tree

Location is a self-referential model with three levels: city, thana, area. Two methods carry the weight:

- ancestors() — walks up via parent, returning the chain root-first.
- descendant_ids() — walks down breadth-first, collecting every id beneath a node.

Together they implement the matching rule that makes search feel right. A provider registers coverage for Dhanmondi (a thana). A customer searches from Kalabagan (an area inside Dhanmondi). The search expands the customer's location into descendant_ids() + ancestors(), filtering out the city level, so the Dhanmondi provider matches. Without the ancestor half, a provider covering a whole thana would be invisible to anyone searching a specific street in it.

The city level is excluded from the ancestor expansion deliberately. Including it would mean anyone who registered 'Dhaka' matched every search in the city, which is how coverage spam starts. For the same reason set_service_areas refuses to store a city-level area at all.

6.3 The check constraint on hierarchy

```
models.CheckConstraint(  
    condition=models.Q(level="city") | models.Q(parent__isnull=False),  
    name="location_non_city_requires_parent",  
)
```

An orphaned thana is unreachable from any tree walk and would silently never match a search. The database refuses to store one.

7. Providers: offerings, coverage, hours and verification

backend/apps/providers/. The largest app. It owns what a provider sells, where, when, and the documents that prove who they are — plus the search that ranks them.

7.1 The four things a provider configures

Model	Holds	Constraint worth noting
ProviderService	One service this provider offers, with their price and estimated duration.	Unique on (provider, service) — you cannot list the same service twice at two prices.
ServiceArea	One location covered, with an optional travel surcharge.	Unique on (provider, location). City level rejected in the service layer.
Availability	A recurring weekly window, e.g. Tuesday 09:00–17:00.	Check constraint that end is after start; unique on (provider, weekday, start_time).
AvailabilityException	One specific date overriding the weekly pattern — a day off, or extra hours on a normally closed day.	Unique per (provider, date). A check constraint forces hours to be present when <code>is_available</code> is true.

How availability is resolved

`resolve_availability()` encodes the precedence in six lines: if an exception exists for that date it wins outright — either 'closed' (empty list) or the exception's own hours. Only when no exception exists does the weekly pattern for that weekday apply. Exceptions are absolute, not additive, which is what a provider intends when they mark a day off.

Overlap detection when saving a weekly schedule uses the standard interval test — two windows overlap when `start < other_end` and `other_start < end` — and the whole set is validated before anything is written, so a partially-saved schedule is impossible.

7.2 Verification: the document pipeline

Four document types map onto three verification flags:

Documents approved	Flag set	Trust points
NID front and NID back	<code>identity_verified</code>	40
Trade certificate	<code>skill_verified</code>	20
Address proof	<code>address_verified</code>	10
(phone OTP, from the accounts app)	<code>phone_verified</code>	30

Note that identity requires *both* NID sides. `_sync_verification_flags()` recomputes all three flags from scratch from the set of currently-approved documents, rather than incrementally toggling them. That means revoking a document correctly un-verifies the provider, which an incremental approach would get wrong.

Every approval or rejection also queues a trust recompute, because verification is 20% of the score.

7.3 Document storage is deliberately not public

This is one of the strongest security stories in the project and worth walking an interviewer through slowly.

National ID scans must never be reachable by URL. `VerificationDocument.file` uses a separate storage backend, `PrivateMediaStorage`, which writes to `PRIVATE_MEDIA_ROOT` — a directory outside anything the web server serves. Its `url()` method does not return a path; it raises:

```
def url(self, name):
    raise NotImplementedError(
        "Private media has no public URL. Serve it through an "
        "authenticated view."
    )
```

So if anyone ever writes a serializer that tries to expose the file URL, the code crashes loudly in a test rather than quietly leaking identity documents. The failure mode is an exception, not a breach.

Admins view documents through `GET /api/v1/admin/verifications/{id}/file/`, which is `IsAdminUser`-gated and streams the file back with four deliberate headers:

Header	Purpose
Cache-Control: no-store	The document never lands in a browser or proxy cache.
X-Content-Type-Options: nosniff	The browser must not second-guess the declared content type.
Content-Security-Policy: default-src 'none'; sandbox	Even if a malicious file somehow rendered, it can load nothing and run nothing.
Content-Disposition: inline	The admin sees it in-browser rather than downloading NID scans onto their laptop.

Content type is derived from a whitelist dictionary keyed by extension, defaulting to `application/octet-stream` — never from the uploaded file's own claimed type.

Retention

On review, `purge_after` is set to 90 days out. The `purge_documents` job then deletes the file bytes while keeping the decision record. The platform retains proof that identity was verified and by whom, without retaining the NID scan indefinitely. That is data minimisation, and it is a good answer to any privacy question.

7.4 Upload validation: magic bytes, not extensions

`VerificationUploadSerializer.validate_file` reads the first 16 bytes of the upload and checks them against the real file signature for the claimed extension:

```
MAX_BYTES = 5 * 1024 * 1024
FORMATS = (
    ({"jpg", "jpeg"}, b"\xff\xd8\xff"),
    ({"png"}, b"\x89PNG\r\n\x1a\n"),
    ({"pdf"}, b"%PDF-"),
)
```

Both the extension and the content must agree. An HTML file renamed to `.jpg` — the classic stored-XSS vector — fails, because its bytes do not begin with the JPEG signature. A real JPEG renamed to `.pdf` also fails, because the pairing is checked, not just membership.

This was a real M10 finding

The original implementation checked only the extension. The security review caught it, and this is worth saying out loud in an interview: 'the first version trusted the extension; the M10 review found it and now the magic bytes are checked.' Describing a bug you found in your own code reads as maturity, not weakness.

7.5 Provider search

apps/providers/search.py. One function, `search_providers()`, builds a single queryset. It measures 100 ms at p95 against 10,018 providers.

The base set

Before any filter: accepting work, user active, not suspended, and not in the `under_review` tier. That last exclusion is a product decision with teeth — a provider under investigation is not merely ranked lower, they are removed from discovery entirely and cannot receive broadcast requests.

Why EXISTS instead of a join

Filtering by service could be written as a join to `ProviderService`. It is written as `Exists(...)` instead, for two reasons: a join multiplies rows when a provider offers several matching services, forcing a `DISTINCT` that costs a sort; and PostgreSQL can short-circuit an `EXISTS` as soon as one row matches.

The matched-price subquery

Sorting by price is not sorting by a column — a provider has many prices. A correlated Subquery annotates each provider with the *lowest price among the services that matched this search*. A plumber whose drain-cleaning is cheap but whose bathroom installation is expensive sorts by the one the customer actually searched for. This is the query most likely to be asked about in an interview, and being able to explain why it is a subquery rather than an aggregate is the whole answer.

Ordering

ordering	Sort	Tie-breaks
-trust_score (default)	Trust descending.	identity_verified, then pk.
price	Matched price ascending, with Coalesce to 0 for nulls.	identity_verified, trust, pk.
distance	Manhattan distance in degrees from the search origin.	trust, pk.

Every ordering ends in `pk`. Without a unique final tie-break, PostgreSQL may return rows in a different order between two queries with the same `ORDER BY`, which makes pagination duplicate and drop rows. That is a classic, real bug and the `pk` at the end is the fix.

Distance uses $\text{abs}(\Delta\text{lat}) + \text{abs}(\Delta\text{lon})$ rather than a proper haversine. Within one city the error is tiny, it needs no PostGIS extension, and it stays inside the database where the sort happens. The honest framing is: it is an approximation chosen deliberately, and PostGIS is the upgrade path if the product ever goes national.

Malformed filters degrade rather than fail

`?min_trust=abc` returns a broader result set, not a 400. For a public search endpoint a bad query parameter — usually from a stale bookmark or a crawler — should show results, not an error page.

8. The trust engine

`backend/apps/trust/`. This is the heart of the project and the thing to spend the most interview time on. It is deliberately split in two: `factors.py` is pure arithmetic with no Django imports at all, and `engine.py` connects it to the database.

Why that split matters

`factors.py` imports only `math` and `dataclasses`. Every formula can be unit-tested with a plain `dataclass` and no database at all, which is why the trust tests run in milliseconds and why `docs/verify_trust_math.py` can be a standalone reference implementation that reproduces the same numbers.

8.1 The six factors

Factor	Weight	Question it answers
F1 Verification depth	20%	Has this person proven who they are?
F2 Job volume	15%	Is there enough evidence to judge them?
F3 Completion reliability	20%	When they commit, do they finish?
F4 Cancellation discipline	15%	How often do they break a commitment, and how badly?
F5 Responsiveness	10%	How long does a customer wait for an answer?
F6 Review quality	20%	What do verified customers say?

Each returns 0–100. The weighted sum is the base score; penalties are subtracted after; the result is clamped to 0–100.

8.2 F1 — Verification depth

Simple additive points: phone 30, identity 40, skill 20, address 10. The weighting is intentional. Identity is worth more than a trade certificate because knowing who someone is matters more for accountability than knowing they passed a course. A fully verified provider scores 100; phone-only scores 30.

8.3 F2 — Job volume, logarithmic

```
def f2_volume(inputs):
    completed = inputs.jobs_completed
    if completed <= 0:
        return 0.0
    raw = 100.0 * log(1 + completed) / log(1 + 100)
    return min(raw, 100.0)
```

Logarithmic with saturation at 100 jobs. The shape is the point: going from 1 to 5 jobs moves the factor about 26 points; going from 50 to 100 moves it about 15. Early evidence is worth much more than late evidence, because the first few jobs tell you most of what you will ever learn, and farming hundreds of trivial jobs to inflate the score has sharply diminishing returns.

8.4 F3 — Completion, with Bayesian smoothing

```
k, mu = 10, 0.90
return 100.0 * (jobs_completed + k * mu) / (jobs_accepted + k)
```

This is the single most important formula in the project. The naive version is completed / accepted, and it is badly broken: a provider who accepted one job and finished it scores 100% — a perfect score on one data point.

The smoothing adds a prior: ten imaginary jobs at the 90% platform average. A new provider therefore starts near 90 and moves toward their real rate as evidence accumulates. Work it through:

Real record	Naive	Smoothed
1 of 1 completed	100.0	90.9 — good, not proven
5 of 5 completed	100.0	93.3 — earning it
50 of 50 completed	100.0	98.3 — genuinely proven
0 of 1 completed	0.0	81.8 — one bad job is not a verdict

It is symmetric, and that symmetry is the fairness argument: it protects a newcomer from one unlucky job as much as it stops one lucky job looking like a track record. $k=10$ is the strength of the prior — how many real jobs it takes before the provider's own record outweighs the platform average.

8.5 F4 — Cancellations, damage-weighted and decayed

Two providers each cancel 10% of jobs. One always gives two days' notice; the other no-shows. Treating those the same is wrong, so each cancellation is weighted by how much damage it did:

Notice given	Weight	Why
24 hours or more	1.0	Annoying; the customer can rebook.
4 to 24 hours	2.0	The day is disrupted.
Under 4 hours	4.0	The customer has taken time off work.
No-show	6.0	They waited and nobody came.

Each is then multiplied by exponential time decay with a 180-day half-life — $0.5^{(days_ago / 180)}$ — so a cancellation from six months ago counts half, and one from a year ago counts a quarter. People improve, and a score that never forgives gives nobody a reason to.

The weighted total is divided by jobs accepted, capped at 1.0, and then raised to the power 1.5:

```
rate = weighted / float(jobs_accepted)
return 100.0 * (1 - min(rate, 1.0)) ** 1.5
```

The exponent makes the penalty convex. A 10% weighted rate costs about 15 points, but a 30% rate costs about 41 — more than double, for triple the rate. Occasional cancellation is human; habitual cancellation is a different kind of provider, and the curve says so.

8.6 F5 — Responsiveness

A piecewise-linear curve over the median response time:

Median response	Score
5 minutes or less	100
5 to 60 minutes	100 down to 70
1 to 4 hours	70 down to 40

Median response	Score
4 to 24 hours	40 down to 0
Over 24 hours	0

Two guards matter. Fewer than five recorded responses returns a neutral 60 rather than a real score, because a median of two data points is noise. And the median is used rather than the mean, over the last 30 responses only, so one holiday does not define a provider forever.

8.7 F6 — Review quality

Bayesian smoothing again, this time on a 1–5 scale with a prior of 4.2 at strength 5, and each review additionally weighted by 365-day time decay:

```
weighted_sum = sum(rating * decay(days_ago, 365) for ...)
weighted_count = sum(decay(days_ago, 365) for ...)
bayes = (weighted_sum + 5 * 4.2) / (weighted_count + 5)
return 100.0 * (bayes - 1) / 4.0
```

The final line rescales 1–5 to 0–100 linearly, so a 1-star average maps to 0 rather than 20. Note that the decay applies to both numerator and denominator — old reviews lose influence rather than being deleted, which is mathematically the right way to age evidence.

Only reviews that are published and not hidden are gathered, which is what makes the double-blind mechanism and admin moderation actually affect the score.

8.8 Penalties, applied after the weighted sum

Condition	Penalty
An upheld dispute in the last 90 days	–15
Two or more upheld disputes in 180 days	–25
A no-show in the last 30 days	–10
Currently under investigation	–20

These stack, and they are subtracted *after* the weighted sum rather than folded into a factor. That ordering is the whole point: if a dispute were one component of a weighted average, a provider with excellent numbers elsewhere would dilute it to almost nothing. A flat subtraction cannot be diluted. One serious incident costs the same fifteen points whether you are excellent or mediocre.

8.9 The worked example that sells the whole idea

Two providers the seeded data actually produces:

	Kamal	Shakib
Star rating	4.9	4.8
Jobs completed	4 of 4	18 of 30
Cancellations	0	12 (3 under 4h notice)
Median response	8 minutes	6 hours
Verification	Full (NID + trade cert)	Phone only

	Kamal	Shakib
Trust score	84.19	53.48

On any conventional marketplace these two look identical — 4.9 against 4.8. Shakib abandons 40% of the jobs he accepts and takes six hours to reply, and his review score is the *highest* of his six factors, which is precisely the one signal a star rating would have shown you. That is the pitch in two sentences, and both numbers are reproduced exactly by the test suite.

8.10 Tiers

Tier	Requires
New	Fewer than 3 completed jobs. Shown honestly as new.
Rising	3+ jobs, any score below the Established bar.
Established	Score ≥ 70 , 10+ jobs, and identity verified.
Trusted Pro	Score ≥ 85 , 25+ jobs, and identity verified.
Under review	Set by an admin. Overrides everything; removes the provider from search and from broadcast requests.

Identity verification is a hard gate on the top two tiers, not a contributor. No amount of good behaviour reaches Established while anonymous. Tiers exist because ‘84.19’ means nothing to a customer at a glance while ‘Trusted Pro’ does; the number is there for anyone who wants to look closer.

8.11 Snapshots: the audit trail

Every recompute writes a TrustSnapshot: the score, the tier, the full factor breakdown as JSON, the algorithm version, and the trigger that caused it. The model extends `AppendOnlyModel`, so it cannot be edited or deleted.

Seven triggers are recorded: booking completed, booking cancelled, review changed, verification decided, dispute changed, nightly batch, and manual. This means a provider who asks ‘why did my score drop on Tuesday?’ gets a real answer, and an admin reviewing an appeal can see exactly what the inputs were at the time.

`algo_version` is stored on every snapshot. If the weights change to v1.1, historical snapshots remain interpretable under the version that produced them. This is unglamorous and it is exactly the kind of thing a senior engineer notices is present.

8.12 When trust is recomputed

Event-driven plus nightly. The event-driven half uses a pattern worth explaining precisely:

```
transaction.on_commit(
    lambda: engine.recompute(provider_id, trigger=mapped)
)
```

`on_commit` defers the recompute until the surrounding transaction has actually committed. Two reasons. First, if the booking transaction rolls back, no snapshot is written for an event that never happened. Second, the recompute reads the provider’s counters — running it inside the same transaction could read a state the rest of the world never sees.

`recompute()` itself takes `select_for_update` on the provider row, so two bookings completing simultaneously cannot interleave and write inconsistent snapshots.

The nightly batch exists because **time decay moves even when nothing happens**. A cancellation ages, a review ages, and the score must reflect that. It iterates with `.values_list('pk', flat=True).iterator()` — ids only, streamed — so memory stays flat at ten thousand providers. It measures 175 seconds against a 15-minute target.

8.13 Anti-gaming, summarised

Every formula choice is a defence. Worth having this list ready as one answer:

Attack	Defence
Create an account, do one perfect job, look flawless	Bayesian priors on F3 and F6; the New tier below 3 jobs; logarithmic F2.
Farm many tiny jobs to inflate volume	Logarithmic F2 saturating at 100 jobs.
Accept everything, deliver selectively	F3 uses accepted as the denominator, not completed.
Cancel late but often, hiding behind good reviews	F4 damage weighting up to 6×, convex exponent 1.5.
Wait out a bad incident	Decay exists but is slow (180 and 365-day half-lives), and penalties sit outside the weighted sum.
Fake reviews from friends	A review requires a completed booking, which requires an accepted request and a confirmed completion; reviews are double-blind and admins can hide them.
Stay anonymous while scoring highly	Identity verification is a hard gate on Established and Trusted Pro.

9. Bookings: the request and job lifecycle

backend/apps/bookings/. Four models and a state machine. This is where most of the concurrency thinking lives.

9.1 The four models

- **ServiceRequest** — a customer asking for work. States: open, accepted, withdrawn, expired. Kinds: broadcast or direct.
- **RequestPhoto** — up to five photos of the problem.
- **ProviderResponse** — one provider's accept or decline, with response_seconds recorded. Unique per (request, provider).
- **Booking** — created when a provider accepts. One-to-one with the request.
- **BookingEvent** — append-only log of every transition: from state, to state, actor, actor user, reason, metadata.

9.2 The state machine

In state_machine.py, transitions are a single dictionary. Being able to point at one 25-line file and say 'every legal move in the system is here' is worth a lot in an interview.

From	May move to
scheduled	in_progress, cancelled_customer, cancelled_provider
in_progress	awaiting_confirm, cancelled_customer, cancelled_provider
awaiting_confirm	completed, disputed
completed	disputed
cancelled_customer	— terminal
cancelled_provider	— terminal
disputed	completed (after resolution)

Three design points to call out:

- 1 **A cancellation is final.** Both cancelled states have empty transition sets. Undoing a cancellation would mean the provider's cancellation counter and the customer's expectations had already diverged from reality — the correct response is a new request, not a resurrection.
- 2 **completed can still become disputed.** Bad work is often only discovered later, and auto-confirmation after 72 hours means 'completed' sometimes means 'nobody objected'.
- 3 **The state machine is enforced in one place.** _transition() calls assert_can_transition before any write, so no service function can move a booking illegally even by mistake.

One transition helper, one event log

Every state change funnels through _transition(), which validates, sets the state, applies any extra field updates, saves with an explicit update_fields, and writes the BookingEvent. Because there is exactly one path, the audit log cannot have gaps — a developer would have to deliberately bypass the helper to create one.

9.3 Who is eligible for a broadcast request

`matching_provider_ids()` is the matching engine. For a direct request it is exactly one provider. For a broadcast it is every provider who:

- has a service area in the requested location's expanded tree (descendants plus non-city ancestors),
- actively offers that specific service,
- is accepting work,
- has an active, unsuspended user account,
- and is not in the `under_review` tier.

The same function answers both 'who should see this?' and 'may this provider respond?' via `can_provider_respond()`. One definition, so the inbox and the authorisation check can never disagree — which is the kind of drift that causes real security bugs.

9.4 The responsiveness denominator

When a request is created, `_bump_requests_received()` increments `requests_received` for every matching provider with a single atomic `F()` update. This is the denominator of F5: a provider who ignores requests is measured, not merely absent from the data. Without it, ignoring every request would look identical to never having been asked.

9.5 Concurrency: the double-accept problem

Two providers hit 'accept' on the same broadcast request at the same millisecond. Both must not get a booking. The fix is in `respond_to_request()`:

```
@transaction.atomic
def respond_to_request(*, request_id, provider_id, accept, ...):
    request = (
        ServiceRequest.objects
        .select_for_update()
        .select_related('service', 'location', 'customer')
        .filter(pk=request_id)
        .first()
    )
```

`select_for_update()` takes a row-level write lock in PostgreSQL. The second provider's transaction blocks at that line until the first commits, then re-reads the row, finds `state == 'accepted'`, and gets a clean `request_closed` error. No race, no duplicate booking, no application-level retry logic.

The same pattern guards the batch jobs. `expire_stale_requests()` and `auto_confirm_bookings()` both iterate loosely, then re-lock and re-check state inside a per-row transaction — so two cron runs firing simultaneously cannot double-process the same row. This is exactly the idempotency rule from section 4.2 in practice.

9.6 What happens on completion

`confirm_completion()` is the busiest moment in the system. In one atomic transaction it:

- 1 locks the booking row,
- 2 transitions `awaiting_confirm` → `completed`, recording the confirmed price,
- 3 writes the `BookingEvent`,
- 4 increments `jobs_completed` atomically,
- 5 calls `record_completion_payment()`, which creates the `Payment` and three `LedgerEntry` rows,
- 6 and queues the trust recompute for after the commit.

All of it is one transaction, so a crash halfway cannot leave a completed booking with no payment record or an incremented counter with no booking.

9.7 Two timers

Timer	Window	Effect
Request expiry	24 hours	An unanswered open request becomes expired. The customer is not left watching a dead request forever.
Auto-confirm	72 hours	A booking left in <code>awaiting_confirm</code> is confirmed by the system at the provider's stated final price, with <code>auto_confirmed=True</code> and a system actor on the event.

Auto-confirm exists because a provider should not go unpaid and un-credited because a customer forgot to tap a button. The flag records that it was automatic, so the data can distinguish 'the customer confirmed' from 'nobody objected' — which is why `completed` → `disputed` stays legal.

10. Reviews and payments

10.1 Why double-blind

In a one-sided system the second reviewer sees the first review before writing theirs. That produces two well-documented distortions: retaliation (a provider who got 2 stars gives the customer 1 star) and reciprocity (both sides quietly inflate to keep the peace). Both destroy the signal the trust engine depends on.

ServoraBd hides both reviews until either both are submitted or 14 days pass. The mechanism is a `published_at` timestamp that is null until reveal, plus a `reveal_deadline`.

```
def _maybe_reveal(booking):
    review = Review.objects.filter(booking=booking).first()
    rating = CustomerRating.objects.filter(booking=booking).first()
    if review is None or rating is None:
        return False
    # both present -- publish both, recompute trust
```

Called after both `create_review` and `rate_customer`, so whichever side finishes second triggers the reveal. The `reveal_reviews` job handles the timeout case hourly: after 14 days the review publishes even if the other side never responded, so one silent party cannot suppress feedback forever.

Crucially, **only published, unhidden reviews feed F6**. An unpublished review is invisible to the trust engine, so a provider cannot infer their rating from a score change before the reveal.

10.2 The other review rules

Rule	Value	Reason
Review window	30 days	After a month, memory is unreliable and the job is stale.
Edit window	24 hours	Correcting a typo or a hasty rating is fair; rewriting history weeks later is not.
Edits are logged	always	<code>ReviewEdit</code> is append-only and stores the previous rating and comment.
Provider reply	one per review	Right of reply, not a comment thread. Only after publication.
Admin hide	reason required	Hiding removes it from F6 and recomputes trust immediately.

A review requires a COMPLETED booking and is one-to-one with it. There is no way to review a provider you never hired, which is what makes fake-review farming expensive: an attacker would need a real request, a real acceptance and a real confirmation for every fake review.

10.3 Payments: cash, honestly modelled

Phase 1 is cash on completion. There is no payment gateway, and the model is honest about that — but it is built so that adding one is additive: `Payment.Method` already has `bKash`, `Nagad` and `card`, and there are `gateway`, `gateway_reference` and `gateway_payload` fields waiting.

The two-sided amount check

The provider records what they charged; the customer confirms what they paid. Both numbers are stored:

- If they match → the payment settles automatically.

- If they differ → status becomes `flagged`, `flagged_at` is stamped, a warning is logged, and **no ledger entries are written** until an admin resolves it at a decided amount with a mandatory note.

This is the right shape for a cash marketplace. The platform cannot observe the transaction, so it records both claims and escalates disagreement rather than silently trusting either party.

The three ledger entries

A settled 1,000 BDT cash job at 12% commission writes:

Kind	Amount	Meaning
earning	+1000.00	The job was worth this.
commission	−120.00	The platform's 12% cut.
cash_retained	−1000.00	The provider already holds the cash.

Balance: −120.00. A **negative balance means the provider owes the platform** its commission, which is exactly right for cash — the money never passed through the platform, so commission is a receivable, not a deduction. When the provider pays it, a settlement entry of +120.00 brings the balance to zero.

`record_provider_settlement()` refuses a settlement larger than the outstanding payable, so the ledger cannot be pushed into a fictitious credit.

Money correctness

- **Decimal, never float.** Every amount is a `DecimalField`, and `money()` quantises to two places with `ROUND_HALF_UP` — banker's rounding would surprise users.
- **The commission rate is copied onto the Payment row.** If the platform rate changes from 12% to 15%, historical payments keep the rate they were settled at. Reading it live from settings would retroactively rewrite history.
- **Ledger entries are append-only.** A correction is a new adjustment entry, never an edit. That is how real accounting works.
- **Money is a string end to end in the frontend.** It is formatted for display and never added, subtracted or rounded in JavaScript, where `0.1 + 0.2 !== 0.3`.

Two database constraints doing real work

```
# at most one cash payment per booking
UniqueConstraint(fields=['booking'],
                 condition=Q(method='cash'),
                 name='one_cash_payment_per_booking')

# at most one earning / commission / cash_retained per booking
UniqueConstraint(fields=['booking', 'kind'],
                 condition=Q(kind__in=['earning', 'commission',
                                       'cash_retained']),
                 name='one_accrual_of_each_kind_per_booking')
```

These are partial unique indexes. Even if a bug called `record_completion_payment` twice, the database refuses the double-accrual. Payouts and adjustments are deliberately excluded from the second constraint, because a provider can legitimately be paid or adjusted more than once.

Reconciliation

`reconcile_provider()` walks every completed booking and checks that a payment exists, that the earning entry equals the settled amount, and that the commission entry equals the commission. The

`reconcile_earnings` job runs it nightly. The README's claim that 'every provider's ledger reconciles against their bookings' is this function, asserted in the test suite.

11. Operations, scheduled work and security

11.1 The seven scheduled jobs

There is no Celery and no Redis. Recurring work is Django management commands run by cron or Windows Task Scheduler. The schedule is declared as data in `apps/operations/jobs.py`:

Job	Cron	What it does
<code>expire_requests</code>	<code>*/15 * * * *</code>	Closes requests nobody answered within 24 hours.
<code>auto_confirm</code>	<code>5 * * * *</code>	Confirms jobs the customer left unconfirmed for 72 hours.
<code>reveal_reviews</code>	<code>10 * * * *</code>	Publishes reviews whose 14-day double-blind window closed.
<code>recompute_all_trust</code>	<code>0 3 * * *</code>	Refreshes every score so time decay keeps moving.
<code>reconcile_earnings</code>	<code>30 3 * * *</code>	Checks every ledger against its payments.
<code>purge_otps</code>	<code>*/10 * * * *</code>	Deletes spent and expired verification codes.
<code>purge_documents</code>	<code>0 4 * * *</code>	Deletes verification files past their 90-day retention.

Because the schedule is data rather than a crontab file, `manage.py crontab` can print the install lines, and the health endpoint can compare each job's last run against its declared interval.

Every run is recorded

`ScheduledCommand` is a base class every job inherits. It creates a `JobRun` row on start, marks it succeeded with a row count on completion, or failed with the exception type and message, then prunes runs older than 30 days.

`GET /api/v1/admin/jobs/` (admin only) reports each job's last status, last success, whether it is **overdue** (no run within twice its interval) and whether it is **stuck** (still marked running long after it should have finished). This is the answer to 'how do you know cron is still working?' — a cron job that silently stops is one of the classic production failure modes, and here it is visible.

Why no Celery

Phase 1 load does not need it, and a queue is not free: a broker to run, a worker to supervise, dead-letter handling, and a whole class of 'the task ran but the transaction had not committed' bugs. The PRD records the thresholds that would justify adopting one, and the three service-layer rules from section 4.2 keep the migration to one line per call site. That is the complete answer: not 'we did not need it' but 'here is when we would, and here is what we did so that day is cheap.'

11.2 The rate-limit bypass, and how it was closed

This is the best security story in the project. Learn it verbatim.

DRF identifies a client for throttling by reading `X-Forwarded-For`. Behind a proxy that is necessary — otherwise every request appears to come from the proxy. But `X-Forwarded-For` is a request header, and **anyone can send one**. With the default configuration, an attacker sends a different forged value on every login attempt and each one looks like a new client. The five-per-15-minutes login limit becomes unlimited, and so does the OTP limit.

The fix is `NUM_PROXIES`, driven by the `TRUSTED_PROXY_COUNT` environment variable. It tells DRF how many proxies sit in front of the application, so it counts that many entries from the *right-hand end* of the header — the part written by infrastructure you control — and ignores everything the client prepended.

Deployment	TRUSTED_PROXY_COUNT
Local development, no proxy	0
One Nginx in front of gunicorn	1
Render (one platform router)	1
CDN in front of Nginx	2

Too high is worse than too low

Setting it too low means everyone behind the proxy shares one bucket — annoying but safe. Setting it too *high* means the limiter starts reading attacker-controlled entries, and the bypass is back. When unsure, guess low.

Because this cannot be verified from code alone — it depends on the deployment topology — there is a command for it. `manage.py check_proxy_count <url>` sends two requests to a live deployment, one with a forged `X-Forwarded-For: 203.0.113.7` and one without, and reads back the `X-Client-Ident` header that `ClientIdentHeaderMiddleware` adds when `EXPOSE_CLIENT_IDENT` is temporarily enabled. If the forged address is what the limiter sees, the command fails with instructions.

Writing a tool to verify your own security configuration against a live deployment is unusual in a portfolio project and worth mentioning without being asked.

11.3 The access-control audit test

`apps/common/tests/test_access_control.py` is the test to show an interviewer if you only get to show one. It walks Django's URL resolver at runtime, collects every route under `/api/v1/`, and asserts that:

- 1 every route not on an explicit 15-entry public allowlist returns **401** to an anonymous caller, on both GET and POST;
- 2 every route under `/api/v1/admin/` returns **403** to an ordinary logged-in customer;
- 3 the allowlist names only routes that actually exist, so a deleted route cannot linger as a permanent exemption;
- 4 at least 60 routes are being checked, so the test cannot silently pass by finding nothing.

The value is that it is **generated from the URL conf, not hand-written**. Add a new endpoint and forget its `permission_classes`, and this test fails before the code merges. Most projects protect endpoints by convention and code review; this one protects them mechanically.

11.4 The rest of the security posture

Concern	How it is handled
SQL injection	The ORM parameterises everything. There is no raw SQL in the codebase.
XSS	React escapes by default and there is no <code>dangerouslySetInnerHTML</code> anywhere. Uploads are magic-byte checked so an HTML file cannot be stored as an image.
CSRF	The API is JWT-in-header, not cookie-authenticated, so it is not CSRF-reachable. Django admin keeps its CSRF middleware.
Password storage	Django PBKDF2 with four validators, minimum length 8, plus the common-password and numeric-only checks.
Object-level access	Every service filters by owner in the same query that fetches the row — <code>filter(pk=booking_id, customer_id=customer_id)</code> — so a wrong owner yields 404, not 403. It does not even confirm the object exists.

Concern	How it is handled
Suspended users	Blocked at login (the token serializer refuses) and at every permission class (not <code>user.is_suspended</code>).
Production headers	HSTS one year with preload and subdomains, SSL redirect, nosniff, X-Frame-Options: DENY, secure and HttpOnly cookies, Referrer-Policy: same-origin.
Dependency vulnerabilities	<code>pip-audit</code> and <code>npm audit</code> run in CI on every push. The Django 5.2 LTS upgrade cleared 37 known vulnerabilities.
Deploy checklist	CI runs <code>manage.py check --deploy --fail-level WARNING</code> against the production settings module, so a misconfiguration fails the build.

The 404-not-403 decision

When customer A requests customer B's booking, the API returns 404. Returning 403 would confirm the booking exists, which leaks information — an attacker could enumerate valid booking ids by the difference between 403 and 404. Because ownership is part of the `filter()` rather than a check after fetching, this falls out of the code structure rather than needing discipline.

12. The frontend

React 18.3, Vite 6, plain JavaScript, TanStack Query for server state, Zustand for session state, Tailwind CSS 4. Mobile-first: every screen is designed at 360 px and the end-to-end suite drives the whole product at that width.

12.1 The layer stack

Layer	File	Job
Transport	api/client.js	fetch wrapper: auth header, token refresh, error normalisation.
Normalisation	api/normalize.js	Every API response mapped from snake_case to camelCase.
Endpoints	api/endpoints.js	One function per backend route, returning normalised data.
Server state	TanStack Query	Caching, refetching, loading and error states.
Session state	store/auth.js	Zustand, with the refresh token persisted to localStorage.
Constants	constants/domain.js	Frozen booking states, trust tiers, labels and tones.
Components	components/	UI primitives, Trust, domain widgets, Layout.
Pages	pages/	One file per route; provider / holds the provider side.

12.2 The five decisions worth defending

JavaScript, not TypeScript

A deliberate choice, with three compensating mechanisms rather than a shrug: frozen constant objects so a typo in `BOOKING_STATE.AWAITING_CONFIRM` throws instead of rendering nothing; a normaliser layer so a backend field rename breaks one file and its tests rather than six screens; and `PropTypes` on every component that touches money, state or trust. It is recorded as a decision in PRD §8.6, not an omission.

React pinned to 18.3, not 19

React 19 silently ignores `propTypes` on function components. Since `PropTypes` are how this JavaScript codebase catches a wrong-shaped prop at runtime, upgrading would switch off the type safety without a single warning. This is a genuinely good answer to ‘why are you on an old React?’

The access token never touches localStorage

Only the refresh token and the user object are persisted, via Zustand's `partialize`. The access token lives in memory only. After a page reload there is therefore no access token, so the first authenticated call refreshes *before* it sends rather than going out bare and taking a 401 and a retry.

Single-flight refresh

This is the subtlest piece of frontend code in the project:

```
let refreshInFlight = null;

export function refreshSession() {
  if (refreshInFlight) return refreshInFlight;
  refreshInFlight = send('/auth/refresh/', {...})
    .then(...)
    .finally(() => { refreshInFlight = null; });
  return refreshInFlight;
}
```

Three components mount at once and all three fire a request with an expired token. Without this, three refresh calls go out. The backend rotates and blacklists refresh tokens, so the first succeeds and **the other two present a now-blacklisted token and fail** — logging the user out at random. Holding the in-flight promise makes all three callers await the same refresh. `client.test.js` asserts both this and the refresh-before-first-call behaviour.

Money is a string, everywhere

Amounts arrive as strings, are stored as strings and are formatted for display. JavaScript never adds, subtracts or rounds them, because IEEE-754 doubles cannot represent 0.1 exactly. Any total the UI shows is computed by the backend in `Decimal`.

12.3 Every page, and what it does

Public pages

Route	File	What happens
/	Home.jsx	Landing page. Its hero is pre-rendered to static HTML at build time, which is what gets LCP to 1.58 s. React then hydrates with identical markup, so layout shift stays at zero.
/services	Services.jsx	Category grid, then services within a category. Data comes from <code>useCatalogue</code> , cached by TanStack Query since the catalogue barely changes.
/providers	ProviderSearch.jsx	The filter panel (service, category, location, minimum trust, tier, verified-only, price band, availability date, sort) and the ranked result list. Filters live in the URL query string, so a filtered search is shareable and survives a reload.
/providers/:id	ProviderProfile.jsx	Full profile: trust breakdown, services with price flags, service areas, weekly hours, work photos, published reviews with provider replies. On a phone the trust breakdown sits directly under the name — it is the signature feature and must not be the last thing on screen.
/compare	Compare.jsx	Up to three providers side by side, compared on trust factors rather than price. The comparison list is a separate Zustand store so it survives navigation.
/login, /register	Auth.jsx	Phone or email, password, role choice on registration, then the OTP step. Errors are read from <code>ApiError.fieldError(name)</code> and shown against the right field.

Customer pages

Route	File	What happens
/request-service	RequestService.jsx	The six-step wizard: Service, Where, Problem, When, Who, Confirm. The draft is written to <code>sessionStorage</code> after every step, so a mid-way refresh loses nothing. Time is picked as a named slot (Morning, Midday, Afternoon, Evening) rather than a raw timestamp, then converted to a Dhaka-local ISO window.
/my-requests	Bookings.jsx	Open, accepted, withdrawn and expired requests, with the responses each received and a withdraw action.
/my-bookings	Bookings.jsx	Bookings in every state, filterable. The same <code>BookingList</code> component serves the provider side with different props.

Route	File	What happens
/my-bookings/:id	Bookings.jsx	One booking with its full event timeline, and the actions legal from its current state — confirm, cancel, dispute. The action set is derived from the state constant, so the UI can never offer a move the backend would reject.
/review/:bookingId	ReviewBooking.jsx	Overall rating plus four optional dimensions (punctuality, quality, professionalism, price fairness) and a comment. It states plainly that the review stays hidden until the provider also submits.

Provider pages

Route	File	What happens
/provider/dashboard	Dashboard.jsx	One call to /provider/dashboard/ returns trust, the eligible-request inbox, active bookings and earnings. Deliberately one aggregated endpoint rather than five round trips on the page a provider opens most.
/provider/bookings	Bookings.jsx	The same list and detail components, with provider actions: start, complete with final price, cancel with a reason.
/provider/earnings	Earnings.jsx	Earnings by day, week or month; the ledger with running balance; outstanding commission payable.
/provider/profile	ProfileSetup.jsx	The largest page. Profile fields, service offerings with prices, service areas from the location tree, the weekly availability grid, verification document upload with status, and work photos.
/provider/trust	TrustPage.jsx	The provider's own score, all six factors with weights and contributions, and score history from the snapshots — so a provider can see exactly what to improve.

12.4 Route protection

RequireRole in Layout.jsx wraps every protected route: unauthenticated users are redirected to login, and users without the required profile are sent away. This is **UX, not security** — it stops a customer landing on a broken provider screen. The real enforcement is the backend permission classes, which is why the access-control audit test exists.

Every route except the landing page is lazy-loaded with React . lazy, so a visitor who only looks at the home page never downloads the provider dashboard's code.

12.5 Performance work, and what was measured

PRD §12.1 sets one number: landing-page LCP under 2.5 s on 4G. `npm run perf:lcp` measures the production build under Chrome's Slow 4G preset (562 ms latency, 1.44 Mbps) with a 4× CPU slowdown — the same conditions Lighthouse uses — over five cold loads, and reports the median. Before M10 the landing page measured 2.63 s, just over. It now measures 1.58 s.

Change	Effect
Prerendered landing hero	LCP 2.45 s → 1.6 s. The headline paints as soon as HTML and CSS arrive, before any JavaScript.
useRoutes instead of createBrowserRouter	Main bundle 104 KB → 85 KB gzipped. The data router pulls in loaders, actions and fetchers that nothing used.

Change	Effect
Static header shell in <code>index.html</code>	First paint 2.4 s → 1.4 s on every page. The browser lays out while the JS is still downloading.
<code><main></code> at least one screen tall	Layout shift 0.21 → at most 0.10. The footer no longer sits in view and then gets pushed down.
<code>app.html</code> preloads public chunks	Saves 175–290 ms on Slow 4G, costs 30–80 ms on Fast 4G. Kept, because the people on slow connections are the ones who give up.

Say this about the pages that miss the target

Pages other than the landing page stay above 2.5 s on Slow 4G. Each needs HTML, then the main bundle, then its route chunk, then its data — and at 562 ms per request that chain is about 2.2 s before any work happens. Getting them under the line needs the data in the first response, which means server-side rendering. That is a Phase 2 architectural decision, not a tweak. Being able to say exactly why a target is missed, and what it would cost to hit it, is far stronger than claiming everything is fast.

12.6 Accessibility

PRD §12.4 asks for WCAG 2.1 AA. `e2e/accessibility.spec.js` runs `axe-core` against every page at 360 px — signed out, as a customer and as a provider — including the open filter panel, the expanded trust breakdown and the mobile menu. It finds zero violations.

The first run did not. Grey slate-400 text measures about 2.6:1 against white, well under the 4.5:1 minimum. Body text now uses slate-500 or darker, and slate-400 survives only on disabled controls, which WCAG exempts.

Automated checks cannot see focus, so there are hand-written tests too: a skip link is the first tab stop on every page, and a test asserts it becomes visible when focused, moves focus into the page, and that the next focused element shows an outline. Knowing the limits of your automated tooling is itself a good answer.

12.7 Testing

Layer	Tool	Covers
Unit	Vitest	The normaliser, formatting helpers, and <code>client.js</code> including the single-flight refresh.
Component	Testing Library	Trust badge and breakdown, domain widgets, UI primitives.
End-to-end	Playwright	Both golden paths in Chromium at 360 px: the customer journey (register, verify, search, request, review) and the provider journey (register, set up, accept, complete, get paid).
Accessibility	axe-core in Playwright	Every page in three auth states.

`npm run e2e` starts its own backend on port 8100 and Vite on 5174, runs everything, then deletes the test users it created via `e2e/cleanup.py`. In CI, failures upload Playwright traces as artifacts and are reported as GitHub annotations on the failing line.

13. Deployment and CI

13.1 Two routes, both honest

The site is not deployed yet, and the documentation says so. There are two prepared routes, both checked locally, and each guide marks which parts remain unverified. Saying that plainly is better than an unsupportable claim an interviewer can puncture in one question.

Route	Shape	Best for
Render	Driven by <code>render.yaml</code> : managed PostgreSQL, a web service for the API, cron jobs for the scheduled work, and a static site for the React build.	No Linux administration. The simpler route.
Ubuntu VPS	Nginx → gunicorn via a unix socket, systemd for supervision, cron for scheduled jobs, <code>pg_dump</code> backups with a restore check.	Full control, and it demonstrates the ops knowledge.

13.2 What changes in production settings

- `DEBUG=False`, real `ALLOWED_HOSTS`, SMTP email backend.
- `OTP_ECHO_TO_LOG=False` — codes never reach the log.
- `WhiteNoise` inserted directly after `SecurityMiddleware`, with the compressed manifest static storage for hashed, far-future-cacheable filenames.
- The full HTTPS header set: SSL redirect, one-year HSTS with preload and subdomains, `SECURE_PROXY_SSL_HEADER` so Django trusts the proxy's protocol, `nosniff`, `X-Frame-Options: DENY`, secure cookies.
- Optional S3-compatible object storage, split into two buckets' worth of configuration: public media with `querystring_auth=False`, and private verification documents with signed URLs expiring in 300 seconds.

That storage split is the same public/private separation as local disk, expressed for object storage — the abstraction held when the backend changed, which is the point of putting it behind `STORAGES` in the first place.

13.3 CI

Four GitHub Actions jobs run on every push and pull request:

- 1 **backend** — against real PostgreSQL 18: a `makemigrations --check` so a model change without a migration fails the build; `check --deploy --fail-level WARNING` against the production settings module; then the full suite with `-W error::DeprecationWarning`, so a deprecation becomes a failure now rather than a surprise at the next Django upgrade.
- 2 **frontend** — ESLint with zero warnings allowed, Vitest, and a production build.
- 3 **e2e** — a real backend with seeded demo data, a real Chromium, both golden paths and the accessibility scan. Traces upload on failure.
- 4 **audit** — `pip-audit` and `npm audit`.

14. Interview preparation: how to present this

14.1 The 90-second pitch

Memorise the shape of this, not the words. It works because it leads with a problem, not a stack.

The pitch

'ServorabD is a local services marketplace — electricians, plumbers — for Dhaka. The booking flow is a solved problem, so what I actually built it to explore is the trust model.

Star ratings are broken in two ways. They cluster: almost everyone sits between 4.5 and 5.0, so they carry almost no information. And they only measure jobs that were *completed* — a provider who accepts ten bookings and abandons eight can hold a perfect score.

So instead of one number people give you, I compute six from things the platform observes: verification depth, job volume, completion rate, cancellation discipline, responsiveness and review quality. Completion and reviews use Bayesian smoothing so one good job does not look like a track record. Cancellations are weighted by notice given — an hour before costs four times what two days before costs — and decay with a 180-day half-life.

In my demo data there are two providers rated 4.9 and 4.8. Their trust scores are 84 and 53, because the second one abandons 40% of what he accepts. That gap is the whole project.

It is Django REST Framework and PostgreSQL with a React client, 569 backend tests, and search runs in 100 ms at p95 against ten thousand providers.'

14.2 Five things to steer the conversation toward

- 1 **The trust algorithm.** Bayesian smoothing, logarithmic volume, damage-weighted decay, penalties outside the weighted sum. Every choice has a reason and an anti-gaming purpose.
- 2 **The service layer with its three rules.** IDs not instances, never touch request, idempotent — and the reason: a Celery migration becomes one line per call site.
- 3 **The access-control audit test.** A test generated from the URL conf that fails if any new endpoint forgets its permission class.
- 4 **The rate-limit bypass and its verification command.** A real vulnerability, understood, fixed, and then made checkable against a live deployment.
- 5 **The single-flight token refresh.** A subtle frontend bug — token rotation plus concurrent requests equals random logouts — found, fixed and tested.

14.3 The weaknesses — own them before they are found

Every one of these will be spotted by a good interviewer. Having a considered answer turns each from a weakness into evidence of judgement. The pattern is always the same: acknowledge, explain the trade-off, state the upgrade path.

They will say	You say
'It is not deployed.'	'Correct, and the README says so. Both routes are prepared — a Render blueprint and a full Ubuntu guide with Nginx, gunicorn, systemd, cron and backups — and each marks which parts I have not verified live. I would rather say that than claim production experience I do not have.'

They will say	You say
'No Celery. That does not scale.'	'Right, and it is deliberate. Phase 1 load does not justify a broker and a worker to supervise. The PRD records the thresholds that would, and every service takes IDs rather than instances, never touches request and is idempotent — so adopting one is a one-line change per call site, not a rewrite.'
'No caching layer.'	'Search is 100 ms at p95 against ten thousand providers, so the cache would be solving a problem I do not have yet. The obvious first target is the catalogue, which barely changes; trust scores are already denormalised onto the provider row, which is the caching that actually mattered.'
'Why not TypeScript?'	'A recorded decision, with three things standing in: frozen constants so a typo throws, a normaliser layer so a backend rename breaks one file, and PropTypes on everything touching money, state and trust. On a team I would use TypeScript; I wanted to prove I could hold the invariants without it.'
'Distance is not real geospatial.'	'It is Manhattan distance in degrees. Within one city the error is negligible, it needs no PostGIS extension, and the sort stays in the database. PostGIS with a GiST index is the upgrade if it goes national.'
'Cash only. No payment gateway.'	'Cash is how the market actually works in Dhaka today, so modelling it honestly was the right first step. The model already has bKash, Nagad and card methods and gateway reference fields; the interesting part is the two-sided amount check that flags a mismatch for an admin instead of trusting one side.'
'Trust weights are arbitrary.'	'They are judgement, not data — there is no historical dataset to fit them to. What I did instead is make them explicit and versioned: every snapshot stores its algo_version, so when there is real outcome data, weights can be refitted and old scores stay interpretable. The formula shapes are the defensible part; the exact numbers are a starting point.'
'No notifications.'	'Correct — FR-8 is unbuilt, and the frontend README says the UI was left out rather than faked, because there is no backend to render.'
'Only one region.'	'The location model is a generic self-referential tree with three levels; adding Chittagong is seed data, not a schema change.'

15. Question bank with model answers

15.1 Architecture

Q: Walk me through your architecture.

A: Django REST Framework backend, React client, PostgreSQL. The backend is nine apps by domain, and each app follows the same six-file split: models for shape and constraints, selectors for reads, services for writes and business rules, serializers for validation and shaping, views for HTTP only, permissions for access. A view parses input, checks permission, calls one service and returns a response — if there is an `if` about business rules in a view, that is a bug. The frontend mirrors it: a transport layer, a normaliser so no component ever reads raw API JSON, an endpoint module, and then pages.

Q: Why separate selectors from services?

A: Different concerns. Selectors own read performance — all the `select_related` and `prefetch_related` that stop N+1 queries — and never write. Services own correctness and write. Separating them means I can optimise a query without reading business logic, and change a business rule without breaking a query plan. It also makes the write surface small and easy to audit.

Q: Why nine apps rather than one?

A: They are split on domain boundaries, not layer boundaries, and the dependencies mostly run one way — bookings depends on accounts and catalogue, not the reverse. Where a cycle would appear, the import is deferred inside the function: bookings imports the trust engine inside `_recompute_trust` rather than at module level. The payoff is that I can read the payments app without knowing anything about reviews.

15.2 Database

Q: How do you prevent two providers accepting the same request?

A: `select_for_update()` inside `@transaction.atomic`. The first transaction takes a row-level write lock on the request; the second blocks at that line until the first commits, then re-reads and finds the state is already accepted, and returns a clean domain error. It is pessimistic locking, which is right here because the contention window is milliseconds and the cost of getting it wrong is two providers turning up.

Q: Where are your database constraints, and why not just validate in code?

A: Application validation can be bypassed — a management command, a shell session, a future bulk import. So the invariants that must never break live in the database. A check constraint that a user has a phone or an email. A check constraint that a non-city location has a parent. Partial unique indexes so there is at most one cash payment per booking and at most one earning, commission and cash-retained entry per booking. If a bug ever called the payment service twice, the database refuses rather than double-paying.

Q: How did you handle N+1 queries?

A: In the selectors, and it is measured rather than assumed. Search uses `select_related('user')` for the one-to-one and a `Prefetch` on service areas with their locations and parents, because the result card shows the area name and its parent. The benchmark command times it: 100 ms at p95 against 10,018 providers, against a 400 ms target.

Q: Why denormalise the counters onto ProviderProfile?

A: Because the trust engine reads them for every provider every night. Deriving `jobs_completed` by counting bookings would make the nightly recompute a ten-thousand-provider join. They are maintained with atomic `F()` expressions so concurrent updates cannot lose an increment, and `reconcile_earnings` checks the derived

data against the source nightly. The trade-off is accepted consciously, with a check that catches drift.

15.3 Security

Q: Tell me about a security issue you found in your own code.

A: The best one is the rate-limit bypass. DRF identifies clients for throttling from X-Forwarded-For, but that is a request header anyone can send. With the default configuration an attacker sends a different forged value per login attempt and every one looks like a new client, so the five-per-15-minutes limit is effectively unlimited — and so is the OTP limit. The fix is NUM_PROXIES, set from an environment variable, so DRF counts from the right-hand end of the header and ignores what the client prepended. Because it depends on the deployment topology rather than the code, I also wrote a command that probes a live deployment with a forged header and fails if the limiter sees it.

Q: How do you store national ID documents?

A: On a separate storage backend rooted outside anything the web server serves, and its `url()` method raises `NotImplementedError` rather than returning a path. So if anyone ever writes a serializer that tries to expose it, the code crashes in a test instead of leaking. Admins view them through an authenticated endpoint that streams the file with `no-store`, `nosniff`, and a CSP of `default-src 'none'; sandbox`. Uploads are validated by magic bytes, not extension — that was an M10 finding; the first version trusted the extension, which let an HTML file be stored as a .jpg. And files are purged 90 days after review while the decision record is kept.

Q: How do you know every endpoint is protected?

A: A test proves it rather than a convention. It walks Django's URL resolver at runtime, collects every route under `/api/v1/`, and asserts that anything not on a 15-entry public allowlist returns 401 to an anonymous caller and that every admin route returns 403 to an ordinary customer. It also asserts the allowlist names only routes that still exist, and that at least 60 routes were found, so it cannot pass by finding nothing. Add an endpoint and forget its permission class and CI fails.

Q: Why 404 rather than 403 for someone else's booking?

A: A 403 confirms the object exists, which lets an attacker enumerate valid ids. Ownership is part of the `filter()` that fetches the row, so a wrong owner simply finds nothing. It falls out of the structure rather than needing anyone to remember it.

Q: Explain your OTP implementation.

A: Six digits from `secrets.randbelow`, stored only as a SHA-256 hash of phone plus code plus `SECRET_KEY`. The phone is in the hash so a code for one number cannot verify another; the secret acts as a pepper so a database dump cannot be brute-forced offline. Comparison is `secrets.compare_digest`, constant-time, so response timing does not leak how much of the code was right. Three rate limits: three sends per phone per hour, five verify attempts per code, and a DRF per-IP throttle — per-phone and per-IP are independent axes and you need both. And the attempt counter is committed in its own transaction before the exception is raised, because otherwise the rollback would undo the increment and the attacker would get unlimited guesses.

15.4 The trust algorithm

Q: Why Bayesian smoothing? Explain it to a non-technical person.

A: Without it, a provider who took one job and finished it scores 100% completion — a perfect score on one data point. Smoothing adds ten imaginary jobs at the platform average of 90%, so a new provider starts near 90 and moves toward their real rate as evidence accumulates. One of one becomes 90.9 instead of 100; fifty of fifty becomes 98.3. It is symmetric, which is the fairness argument: it protects a newcomer from one unlucky job exactly as much as it stops one lucky job looking like a record. Non-technically: *we do not trust a small sample, in either direction.*

Q: Someone games your system. How?

A: Every formula is a defence against a specific attack, so let me go through them. Create an account and do one perfect job — blocked by the Bayesian priors and the New tier below three jobs. Farm tiny jobs for volume — F2 is logarithmic and saturates at 100. Accept everything and deliver selectively — F3's denominator is jobs *accepted*, not completed. Cancel late but often and hide behind reviews — F4 weights a no-show six times a two-day-notice cancellation and the curve is convex. Wait out a bad incident — decay is slow, 180 and 365-day half-lives, and penalties sit outside the weighted sum so they cannot be diluted. Fake reviews — a review needs a completed booking, which needs a real request, acceptance and confirmation. The genuinely unsolved one is collusion: two accounts booking each other. That needs graph analysis on the booking network, and it is honestly Phase 2.

Q: Why are penalties applied after the weighted sum?

A: Because a weighted average dilutes. If an upheld dispute were one component of the average, a provider with excellent numbers elsewhere would reduce it to almost nothing. A flat subtraction cannot be diluted — one serious incident costs the same fifteen points whether you are excellent or mediocre. That ordering is a deliberate statement about how seriously the platform takes upheld complaints.

Q: How would you validate that the weights are right?

A: I would not claim they are. They are judgement, because there is no historical outcome data to fit them to. What I did instead is make them explicit, keep them in one dictionary, version them, and store the version on every snapshot — so when there is real data about which providers customers rebook and which generate complaints, I can fit the weights properly and historical scores stay interpretable under the version that produced them. The formula *shapes* are the defensible part; the exact coefficients are a starting point I have made easy to change.

Q: Why recompute nightly if it is event-driven?

A: Because time decay moves when nothing happens. A cancellation ages past its half-life and a review loses weight whether or not the provider does anything. Without the nightly batch, an inactive provider's score would be frozen at whatever it was on their last job. It streams primary keys with `.values_list('pk', flat=True).iterator()` so memory stays flat, and takes 175 seconds for ten thousand providers.

15.5 Frontend

Q: Where do you store the JWT, and why?

A: The access token is in memory only; only the refresh token is persisted to localStorage. That is a considered trade-off rather than the textbook answer. HttpOnly cookies would be safer against XSS, but they bring CSRF back, and the app has no XSS surface: React escapes by default, there is no `dangerouslySetInnerHTML`, and uploads are magic-byte validated. Keeping the access token out of storage limits the window if that reasoning ever turns out to be wrong, and the short 15-minute lifetime limits it further.

Q: Tell me about a hard bug you fixed.

A: Random logouts. The backend rotates refresh tokens and blacklists the old one after rotation, which is good security. But when three components mounted at once with an expired access token, all three fired a refresh. The first succeeded and blacklisted the token; the other two then presented a blacklisted token and failed, clearing the session. The fix is single-flight: hold the in-flight promise in a module-level variable and return it to every concurrent caller, so they all await the same refresh. It is four lines, and the test asserts that two concurrent calls produce exactly one network request.

Q: How did you get LCP under 2.5 seconds?

A: By measuring first. A script runs the production build under Chrome's Slow 4G preset with a 4× CPU slowdown — Lighthouse's mobile conditions — five cold loads, median reported. It started at 2.63 s. Prerendering the landing hero to static HTML at build time took it to about 1.6 s, because the headline paints as soon as HTML and CSS

arrive; React then hydrates with identical markup, so layout shift stays at zero. Switching from `createBrowserRouter` to `useRoutes` cut 19 KB gzipped that was loaders and fetchers nothing used. A static header shell in the HTML moved first paint on every page. Final number is 1.58 s. I also tried preloading the current route's chunk from `main.jsx` and dropped it — only about 40 ms, because the bundle has to be evaluated first.

Q: Why not TypeScript, really?

A: Partly to prove I could hold the invariants without it. The three mechanisms are frozen constant objects so a typo in a state name throws instead of rendering nothing, a normaliser layer so a backend field rename breaks one file and its tests rather than six screens, and `PropTypes` on every component touching money, state or trust. That is also why React is pinned at 18.3 — React 19 ignores `propTypes` on function components silently. On a team, I would use TypeScript.

15.6 Testing and process

Q: What do your 569 tests actually cover?

A: Four layers. Pure-function tests on the trust factors — `factors.py` has no Django imports, so those run in milliseconds against a dataclass. Service tests for business rules, including the concurrency paths. API tests for status codes, permissions and error envelopes. And cross-cutting audits: the access-control test generated from the URL conf, and the ledger reconciliation. The two PRD worked examples are asserted exactly — 84.19 and 53.48 — so a change to any weight fails the suite and tells me the documentation has gone stale.

Q: What would you do differently?

A: Three things. I would write the access-control audit test in the first week rather than during hardening, because it would have caught things earlier and cost nothing. I would deploy at M3 instead of M10 — deploying early surfaces configuration problems while there is little to configure. And I would build the notification app, because several flows have an obvious hole where a notification belongs and users currently have to poll the page.

Q: What is the next thing you would build?

A: Notifications, because it is the largest missing piece and every other feature is degraded without it. Then a real deployment, then a payment gateway now that the model is ready for it. Further out, the interesting problem is collusion detection — the one gaming vector the current algorithm does not address, which needs graph analysis of the booking network rather than another factor.

16. Quick reference

16.1 Numbers to have memorised

Thing	Number
Trust factors	6
Weights	20 / 15 / 20 / 15 / 10 / 20
Completion prior	$k = 10, \mu = 0.90$
Review prior	$k = 5, \mu = 4.2$
Volume saturation	100 jobs
Cancellation half-life	180 days
Review half-life	365 days
Cancellation weights	1× / 2× / 4× / 6× no-show
Cancellation exponent	1.5
Access token / refresh token	15 minutes / 14 days
OTP TTL / sends per hour / attempts	5 minutes / 3 / 5
Login throttle	5 per 15 minutes
Request expiry / auto-confirm	24 hours / 72 hours
Review window / edit window / reveal	30 days / 24 hours / 14 days
Document retention	90 days after review
Commission	12%
Backend tests	569
Search p95 / trust recompute	100 ms / 175 s
Landing LCP, Slow 4G	1.58 s

16.2 Files to be able to open instantly

If they ask about	Open
The trust algorithm	backend/apps/trust/factors.py
How trust reaches the database	backend/apps/trust/engine.py
The booking lifecycle	backend/apps/bookings/state_machine.py
Concurrency	backend/apps/bookings/services.py, respond_to_request
Search performance	backend/apps/providers/search.py
Security auditing	backend/apps/common/tests/test_access_control.py
The rate-limit fix	backend/apps/operations/management/commands/check_proxy_count.py

If they ask about	Open
Private document storage	backend/apps/common/storage.py
Upload validation	backend/apps/providers/serializers.py, VerificationUploadSerializer
Money correctness	backend/apps/payments/models.py and services.py
The token refresh bug	frontend/src/api/client.js
Error handling	backend/apps/common/exceptions.py

16.3 Sentences worth having ready

- 'The booking flow is a solved problem. What I built this to explore is the trust model.'
- 'Star ratings only measure jobs that were completed. A provider who abandons eight of ten can hold a perfect score.'
- '4.9 versus 4.8 on stars; 84 versus 53 on trust. That gap is the project.'
- 'Bayesian smoothing means we do not trust a small sample, in either direction.'
- 'Penalties are outside the weighted sum, because an average dilutes and a subtraction cannot be diluted.'
- 'Every service takes IDs, never touches request, and is idempotent — so adopting a queue is one line per call site.'
- 'The access-control test is generated from the URL conf, so a new endpoint without a permission class fails CI.'
- 'The private storage backend raises on url() rather than returning a path, so a leak is a crash in a test.'
- 'It is not deployed, and the README says so — both routes are prepared and each marks what I have not verified live.'

The habit that makes all of this land

For every answer, give the decision, then the reason, then the trade-off you accepted. 'I used `select_for_update` — pessimistic locking, because the contention window is milliseconds and two providers arriving at one house is worse than a few blocked requests.' That three-part shape is what separates someone who followed a tutorial from someone who made choices.